PERSONAL FINANCE ADVISOR BOT

AI-assisted monthly budgeting and financial guidance

Project Documentation	Developed by
Personal Finance Advisor Bot	Anushka Patil

Repository: https://github.com/patilanushka11/Personal-Finance-Advisor-Bot-anu

The report follows the structure and level of detail of the supplied reference documentation. Technical details are based on the project repository README and the uploaded application screenshot.

Project Documentation

Personal Finance Advisor Bot is a web-based budgeting assistant that helps a user organize monthly income and expenses, select a financial goal, and receive a structured analysis of spending and saving opportunities. The application presents a simple form where the user enters a name, monthly income, expense categories and amounts, and a financial goal.

The project combines a Flask backend with a browser-based HTML, CSS and JavaScript interface. When the user submits the form, the frontend sends the information to the **/analyse** endpoint using a JSON/AJAX POST request. The backend validates the submitted values, prepares a financial prompt, obtains an AI response when Gemini is configured, parses the response, and returns data for display in the interface.

Main objective

The main objective is to make monthly budgeting easier to understand. Instead of manually comparing every expense, users can enter their financial details in one place and receive a summary, category-based spending analysis, and practical saving suggestions.

Key features

- Monthly income and expense entry through a single-page interface.
- Personalized monthly budget and spending analysis by category.
- Selection of a financial goal, such as an emergency fund or investment.
- Balance and saving-rate calculation.
- Three to five saving suggestions based on the submitted information.
- Gemini AI integration, with a deterministic demo fallback when no API key is configured.
- Local storage of analysis records using SQLite and SQLAlchemy.
- Responsive frontend and optional public demonstration through Ngrok.

The application is an educational budgeting assistant. It is not a substitute for professional financial or investment advice.

1. Scenarios

Scenario 1: Entering monthly financial information

The first scenario demonstrates how a user enters their monthly income and regular expenses. The user provides a name and monthly income, selects a financial goal, and adds expense labels with their corresponding amounts. Example categories visible in the uploaded screenshot include rent, food, transport, dining, and entertainment.

Activities

- Activity 1.1: The user opens the Personal Finance Advisor Bot page.
- Activity 1.2: The user enters their name and monthly income in INR.
- Activity 1.3: The user selects a financial goal from the available options.
- Activity 1.4: The user enters expense categories and their amounts.
- Activity 1.5: The user submits the form for financial analysis.

Scenario 2: Analysing spending and savings

After submission, the browser sends the financial information to the Flask backend through the /analyse endpoint. The backend validates the values and prepares the analysis request. The application calculates the balance and saving rate and organizes expense information for category-wise analysis. The resulting information is returned to the frontend for dynamic display.

- Activity 2.1: The frontend packages the form values as JSON.
- Activity 2.2: Flask receives and validates the request.
- Activity 2.3: The backend builds the financial prompt and prepares the analysis.
- Activity 2.4: The response is parsed into structured information.
- Activity 2.5: The interface displays the budget analysis and saving suggestions.

Scenario 3: Receiving AI-assisted saving suggestions

The third scenario demonstrates the AI-assisted guidance feature. When a valid Gemini API key is configured, the backend uses the configured Gemini model to generate natural-language financial analysis. If the key is unavailable, a deterministic local fallback allows the budgeting workflow and interface to be demonstrated. The source indicator distinguishes the Gemini response from the fallback mode.

- Activity 3.1: The user submits monthly income, expenses, and a goal.
- Activity 3.2: The backend creates a prompt using the submitted financial context.
- Activity 3.3: Gemini generates analysis when configured.
- Activity 3.4: The response parser extracts structured output.
- Activity 3.5: The user reviews the analysis and saving suggestions.

2. Application Screenshots

The following screenshots show the Personal Finance Advisor Bot interface, including monthly financial information entry, the generated financial summary, category-wise spending analysis, and saving suggestions.

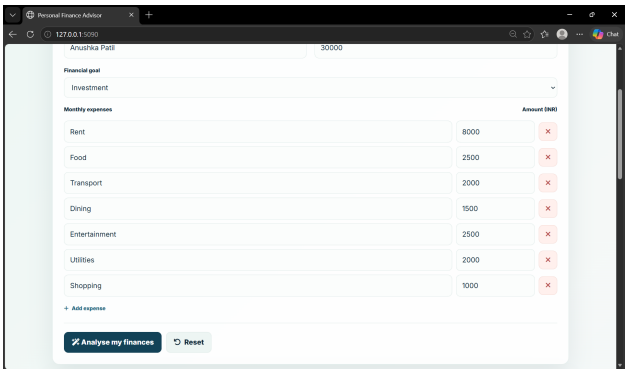


Figure 1: Monthly snapshot form showing the financial goal and monthly expense inputs.

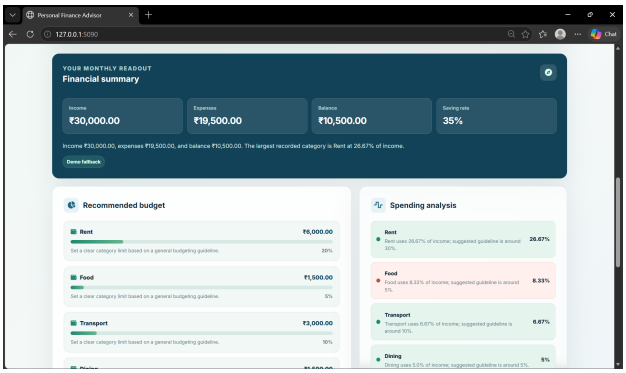


Figure 2: Financial summary, recommended budget, and spending analysis.

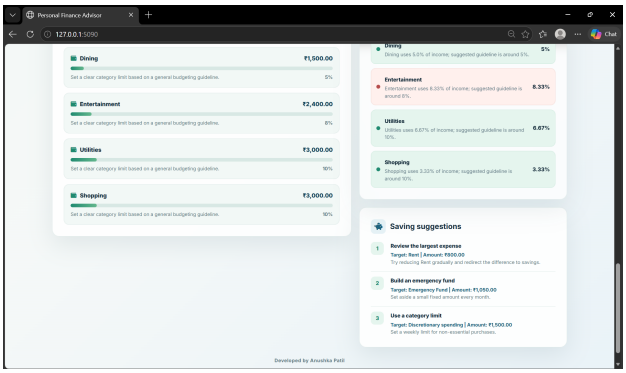


Figure 3: Category-level spending insights and personalized saving suggestions.

3. Technical Architecture

The project uses a compact web application architecture. The browser interface collects user input and communicates with a Flask backend. The backend validates the request, creates a prompt, invokes the configured Gemini service when an API key is available, parses the response, and returns structured results. SQLite persistence is handled through SQLAlchemy.

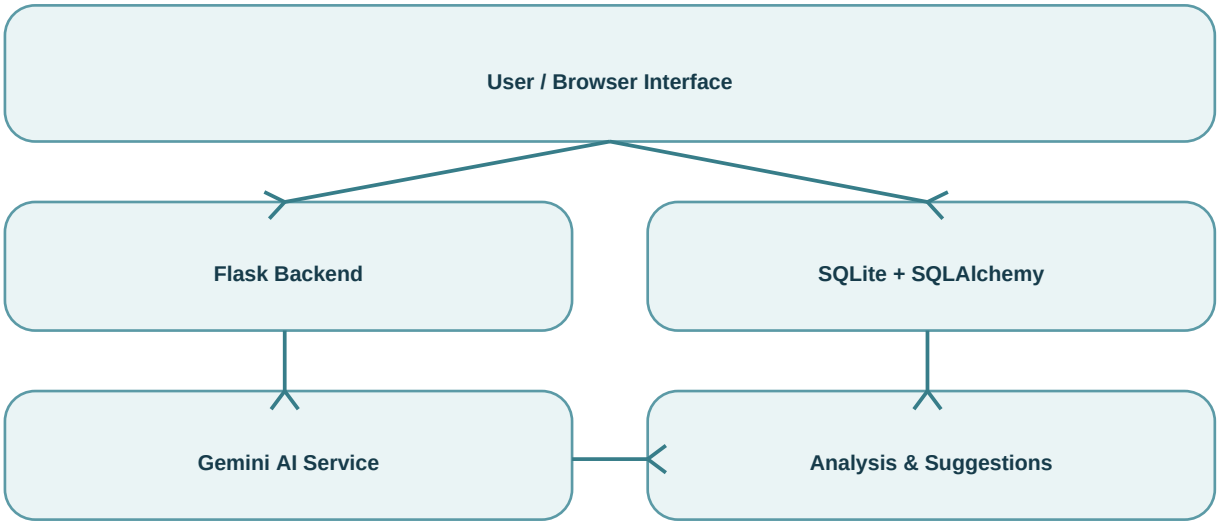


Figure 2: High-level architecture based on the repository documentation.

3.1 Frontend interface

The frontend is implemented as a single-page HTML, CSS and JavaScript interface. It collects the user's name, monthly income, expense details, and financial goal. JavaScript sends the form data to the backend using JSON/AJAX and dynamically renders the analysis, budget information, and saving suggestions.

3.2 Flask backend

The Flask application provides the /analyse POST endpoint. It receives the submitted data, validates the input, builds the financial prompt, coordinates the AI or fallback analysis, parses the result, and sends structured output back to the frontend.

3.3 Gemini AI service

The repository README identifies Gemini gemini-3.8-flash as the configured generative AI model. The API key is supplied through the GEMINI_API_KEY environment variable. The project also includes a local deterministic fallback so that the core demonstration can run without a configured Gemini key.

3.4 Data storage

Analysis records are stored locally in SQLite through SQLAlchemy. The repository documentation states that the application does not currently use Google Sheets.

3.5 Prompt and response processing

The backend uses a build_prompt() helper to prepare the request and an extract_json() helper to parse structured response content. CATEGORY_TIPS and GOAL_DESCRIPTIONS provide supporting category and goal information used by the application.

4. Project Workflow

The development workflow can be organized into six milestones, following the project implementation described in the repository.

Milestone 1: Requirement analysis and setup

Identify the purpose of a personal finance budgeting assistant, define the input fields and desired output, and prepare the Python environment and project dependencies.

Milestone 2: User interface development

Create the single-page form for name, monthly income, financial goal, and expense categories. Style the interface for clear and responsive use.

Milestone 3: Backend development

Implement the Flask application and the /analyse POST endpoint. Receive JSON input and validate the submitted financial values.

Milestone 4: Financial prompt and analysis

Implement prompt construction with build_prompt(), category and goal guidance, balance and saving-rate calculation, and the analysis workflow.

Milestone 5: Gemini integration and response handling

Configure Gemini through GEMINI_API_KEY, call the configured model, parse structured output with extract_json(), and retain a deterministic fallback when the key is absent.

Milestone 6: Storage, testing and deployment

Store analysis records in SQLite through SQLAlchemy, test the form-to-result workflow, and provide local execution plus optional Ngrok-based public demonstration.

Typical request flow

User enters monthly details → Browser sends JSON → Flask validates input → Prompt and financial analysis are prepared → Gemini or local fallback returns a result → Response is parsed → Results are displayed and the analysis record is stored locally.

5. Website Pages / Modules

The repository describes the application as a single-page interface rather than a multi-page financial management portal. Its main functional areas are summarized below.

5.1 Monthly snapshot form

This is the primary input area. It accepts the user's name and monthly income, allows selection of a financial goal, and provides rows for expense names and amounts. The uploaded screenshot shows a clean form layout with INR labels and controls for removing expense rows.

5.2 Expense entry and category analysis

Users enter spending under categories such as rent, food, transport, dining, and entertainment. The application uses the submitted categories to produce a spending analysis and identify areas where spending may be adjusted.

5.3 Financial goal selection

The user selects a financial goal to guide the analysis. The repository identifies goal descriptions through GOAL_DESCRIPTIONS and includes goal selection as one of the application's core features.

5.4 Budget and savings analysis

The application calculates the balance and saving rate from the submitted monthly financial information. It prepares a personalized monthly budget view and presents three to five saving suggestions.

5.5 AI analysis and suggestions

When Gemini is configured, the application uses AI to produce a natural-language explanation based on the user's financial details. When Gemini is not configured, the local fallback provides deterministic analysis for demonstration purposes.

5.6 Local analysis records

The project uses SQLite with SQLAlchemy to store analysis records locally. The README describes this as local storage and explicitly notes that Google Sheets is not currently used.

6. Technology Stack and Project Structure

Component	Technology / implementation
Backend	Python with Flask
Frontend	HTML, CSS and JavaScript
Frontend-backend communication	JSON/AJAX POST request
AI service	Google Gemini API; model named in README: gemini-3.8-flash
Prompt helper	build_prompt()
Response parsing	extract_json()
Category and goal guidance	CATEGORY_TIPS and GOAL_DESCRIPTIONS
Local persistence	SQLite with SQLAlchemy
Public demo option	Ngrok via run_public.py
Local application address	http://127.0.0.1:5090

Repository folder structure

```
Personal-Finance-Advisor-Bot-anu/
■■■ app.py
■■■ run_public.py
■■■ requirements.txt
■■■ .env.example
■■■ README.md
■■■ templates/
■ ■■■ index.html
■■■ static/
■■■ css/style.css
■■■ js/main.js
```

Local setup and execution

1. Create and activate a Python virtual environment.
2. Install dependencies using `pip install -r requirements.txt`.
3. Copy `.env.example` to `.env` and add the Gemini API key if AI responses are required.
4. Run `python app.py`.
5. Open `http://127.0.0.1:5090` in a browser.

For public demonstration, the repository provides run_public.py, which starts the Flask service and creates an Ngrok tunnel. The temporary public URL is available only while the deployment process is running.

7. Testing and Observations

The primary workflow to test is the complete movement of data from the browser form to the backend and back to the results interface. Testing should verify that submitted financial details are accepted, processed, and displayed in a readable format.

Suggested functional test cases

Test case	Expected observation
Valid monthly income and expenses	Form submits and analysis results are displayed.
Different expense categories	Category values are included in the spending analysis.
Financial goal selection	Selected goal is reflected in the generated guidance.
Gemini API key configured	AI response is generated and identified as Gemini AI.
Gemini API key missing	Demo fallback supports the budgeting workflow.
Analysis record storage	Analysis record is saved locally using SQLite/SQLAlchemy.

Security and responsible use

The Gemini API key and Ngrok authtoken should be kept in the private .env file and must not be committed to the public repository. The project README explicitly warns against committing these secrets. The application is intended for educational budgeting assistance and should not be presented as professional financial or investment advice.

Current scope note

The report describes only the functionality and implementation documented in the supplied repository and screenshot. It does not claim bank-account integrations, live market data, investment execution, or professional financial planning features.

8. Conclusion

The Personal Finance Advisor Bot provides a simple way to enter monthly income, organize expenses, select a financial goal, and receive budgeting analysis with saving suggestions. Its browser interface is connected to a Flask backend through a JSON/AJAX request, while the backend coordinates financial analysis, Gemini integration, response parsing, and local record storage.

A useful aspect of the implementation is its fallback behavior. The application can demonstrate its budgeting workflow even when a Gemini API key is not configured. When the key is available, Gemini is used to generate AI-assisted guidance. SQLite and SQLAlchemy provide local storage for analysis records, while Ngrok offers an optional way to expose the running local application for a demonstration.

Overall, the project demonstrates how a lightweight web application can combine user-entered financial data, basic budgeting calculations, AI-generated explanations, and a responsive interface to support better awareness of monthly spending and savings.

Project references

1. GitHub repository: https://github.com/patilanushka11/Personal-Finance-Advisor-Bot-anu
2. Uploaded application screenshot supplied for this report.
3. Personal Finance Advisor Bot README and implementation notes in the project repository.

Project author: Anushka Patil